

LEAK MVP — Detector de Gastos Recurrentes

“Descubrí cuánto dinero estás perdiendo sin darte cuenta.”

Qué es LEAK

LEAK es una aplicación web que te ayuda a descubrir gastos recurrentes (suscripciones, membresías, apps) que podrías estar pagando sin darte cuenta.

No es: Banco, inversión, presupuesto.

Es: Un detector simple y visual de fugas de dinero.

Demo

`https://tu-dominio.com/leak`

Landing → Welcome → Agregar Gasto → Análisis → Resultado → Premium

Estructura del Proyecto

```
LEAK/
├── index.html                # Aplicación principal (SPA)
├── vercel.json               # Configuración Vercel
├──
├── README.md                # Este archivo
├──
├── DOCUMENTACIÓN/
│   ├── LEAK-PUBLICATION-SUMMARY.md    # Resumen técnico
│   ├── LEAK-PUBLICATION-TESTS.md      # 12 tests de validación
│   ├── LEAK-DELIVERY.txt              # Checklist final
│   ├── LEAK-FINAL-REFACTOR.md         # Arquitectura
│   ├── LEAK-VALIDATION-CHECKLIST.md   # Flujo completo
│   ├── LEAK-CHANGELOG.md              # Historial de cambios
│   ├── LEAK-FREE-APP-INTEGRATION.md   # Guía de integración
│   └── LEAK-FREE-APP-FLOW.md           # Flujo visual
```

⚡ Quick Start

Local (sin servidor)

1. Descargar `index.html`
2. Abrir en navegador
3. Funciona 100% offline (usa `localStorage`)

Vercel (recomendado)

1. Crear cuenta en vercel.com
2. Crear nuevo proyecto
3. Subir `index.html` + `vercel.json` en raíz
4. Deploy automático
5. URL: `https://tu-proyecto.vercel.app/leak`

Hosting Estático

1. Subir `index.html` como archivo raíz
 2. Configurar servidor para servir `index.html` en rutas no encontradas
 3. URL: `https://tu-dominio.com/leak`
-

🎮 Cómo Funciona

Flujo de Usuario

LANDING (10 secciones)

↓

"DESCUBRIR MIS FUGAS"

↓

WELCOME

└─ AGREGAR MI PRIMER GASTO

└─ PROBAR CON DATOS DE EJEMPLO

↓

AGREGAR GASTO

- Nombre del servicio

- Precio

- Frecuencia (semanal, mensual, trimestral, anual)
- Categoría
↓
SELECTOR USO
- Nunca
- 1-2 veces/mes
- 3-5 veces/mes
- 6-10 veces/mes
- Más de 10 veces/mes
↓
SELECTOR IMPORTANCIA
- Escala 1-5
↓
ANÁLISIS (1.5s loader)
↓
RESULTADO
├ Si <3 gastos: Primer resultado
└ Si ≥3 gastos: Análisis completo
- Total anual/mensual
- Leak Score (0-100)
- Posibles fugas (>60)
- Ahorro potencial
- Expenses list
↓
"DESBLOQUEAR LEAK PREMIUM"
↓
PREMIUM SCREEN
- 6 features
- "Próximamente" (sin checkout)

Datos Demo

5 gastos precargados (click "PROBAR CON DATOS DE EJEMPLO"):

Servicio	Precio	Frecuencia	Uso	Importancia
Netflix	\$15.99	mes	1-2/mes	2
Spotify	\$11.99	mes	+10/mes	5
Canva	\$15	mes	1-2/mes	1
iCloud	\$2.99	mes	nunca	1
Gym	\$35	mes	3-5/mes	4

Resultados calculados:

- Total anual: **\$971.76**
 - Total mensual: **\$80.98**
 - Leak Score: **61** (dinámico)
 - Posibles fugas: **4** (Netflix, Canva, iCloud, Gym)
 - Ahorro potencial: **\$407.76**
-

Características Técnicas

Tecnología

- **Vanilla JavaScript** (sin frameworks)
- **HTML5 + CSS3**
- **localStorage** (persistencia local)
- **SPA routing** (navegación interna)

Funcionalidades

-  Cálculos 100% dinámicos
-  Leak Score algorithm
-  Posibles fugas filtradas
-  Ahorro potencial calculado
-  localStorage (datos persisten)
-  Tracking de eventos
-  XSS Protection (escapeHTML)
-  Mobile responsive
-  Sticky CTA (mobile)
-  Demo mode

Almacenamiento

```
// localStorage keys
leak_expenses      // Array JSON de gastos
leak_demo_mode     // Boolean (demo activo)
```

Cálculo del Leak Score

```
function calculateLeakScore(expense) {
  let score = 50; // Base

  // Factor costo anual
  if (annualCost > 200) score += 15;
  if (annualCost > 500) score += 15;

  // Factor uso (menor uso = mayor score)
  const usageMap = {
    never: 30,
    rare: 20,
    moderate: 10,
    frequent: 5,
    very_frequent: 0
  };
  score += usageMap[usage];

  // Factor importancia (menor importancia = mayor score)
  score -= (importance * 3);

  return Math.max(0, Math.min(100, score)); // 0-100
}
```

Ejemplo con Netflix (\$15.99/mes, uso 1-2/mes, importancia 2):

```
Base: 50
+ 15 (annual > $200) = 65
+ 20 (uso rare) = 85
- 6 (importance 2 × 3) = 79 ← Score
```

Seguridad

XSS Protection

```
function escapeHTML(value) {  
    const div = document.createElement('div');  
    div.textContent = value;  
    return div.innerHTML; // Escapa < > y caracteres especiales  
}
```

Ejemplo:

```
// Input usuario  
const maliciousInput = '<script>alert("xss")</script>';  
  
// Cómo se guarda  
const escaped = escapeHTML(maliciousInput);  
// Resultado: &lt;script&gt;alert("xss")&lt;/script&gt;  
  
// Se muestra como texto, NO se ejecuta
```



Responsive Design

-  Desktop (1920px+)
 -  Tablet (768px - 1024px)
 -  Mobile (320px - 767px)
 -  No scroll horizontal
 -  Sticky CTA mobile
 -  Touch targets >44px
-



Meta Ads Integration

URL Base

<https://tu-dominio.com/leak>

Con UTM Parameters

https://tu-dominio.com/leak?utm_source=meta&utm_medium=cpc&utm_campaign=leak_disc

Tracking Automático

Eventos que se registran automáticamente:

```
landing_view
landing_cta_clicked
free_app_started
expense_started
expense_added
first_result_viewed
full_analysis_viewed
premium_viewed
expense_deleted
demo_started
```

Ver en **DevTools Console** (reemplazar con GA4/Meta Pixel cuando sea necesario).



Archivos Modificados

index.html

-  Renombrado desde `leak-complete.html`
-  Agregada función `escapeHTML()` (línea 1392)
-  Usada en `leak.name` (línea 1715)
-  Usada en `expense.name` (línea 1735)
-  TODO LO DEMÁS: SIN CAMBIOS

vercel.json

-  NUEVO archivo
 -  Configura rewrite: `/leak` → `/index.html`
 -  Permite SPA routing en Vercel
-



Tests Implementados

12 tests obligatorios — Resultado: 12/12 PASS 

Ver `LEAK-PUBLICATION-TESTS.md` para detalles completos.

1. Landing visible
2. Welcome en click
3. Demo data carga
4. Totales dinámicos
5. Leak Score dinámico
6. Posibles fugas dinámicas
7. Ahorro potencial dinámico
8. Eliminar gasto recalcula
9. Agregar gasto recalcula
10. localStorage persiste
11. Premium funciona
12. XSS bloqueado

Versiones

Versión	Cambios
1.0	Release inicial MVP
-	-

Documentación

- **LEAK-PUBLICATION-SUMMARY.md** — Resumen técnico completo
- **LEAK-PUBLICATION-TESTS.md** — Todos los 12 tests explicados
- **LEAK-FINAL-REFACTOR.md** — Arquitectura y refactorización
- **LEAK-VALIDATION-CHECKLIST.md** — Checklist de validación
- **LEAK-CHANGELOG.md** — Historial de cambios

- **LEAK-FREE-APP-INTEGRATION.md** — Guía de integración
 - **LEAK-FREE-APP-FLOW.md** — Flujo visual del usuario
-

Próximos Pasos

Fase 1: Setup

- Forkear/clonar repo
- Descargar archivos
- Subir a Vercel o hosting

Fase 2: Testing

- Ejecutar los 12 tests
- Validar en mobile
- Verificar localStorage

Fase 3: Launch

- Conectar dominio
- Agregar Meta Pixel (cuando sea necesario)
- Lanzar campaigns Meta Ads

Fase 4: Iteración

- Recolectar data de users
 - Iterar copy/colores
 - Escalar tráfico
-

Licencia

MIT — Usa libremente

Soporte

Problemas comunes:

P: ¿Funciona offline?

R: Sí, totalmente. localStorage funciona sin servidor.

P: ¿Dónde se guardan los datos?

R: En localStorage del navegador. No se envía a servidor.

P: ¿Cómo integro Meta Pixel?

R: Buscar `TU_PIXEL_ID_AQUI` en index.html y reemplazar.

P: ¿Puedo cambiar los datos demo?

R: Sí, editar `loadSampleData()` en index.html

P: ¿Cómo agrego más gastos?

R: El usuario agrega en la app. En código, modificar `sampleExpenses` array.

Status

LISTO PARA PUBLICACIÓN

Todos los archivos están listos para subir a GitHub y deployar.

Versión: 1.0 Final

Fecha: 2026-08-18

Desarrollador: Claude

Proyecto: LEAK MVP